

CasePilot

Autonomous customer
resolution

It investigates like an analyst, acts like an operator, and proves its
work like an auditor.

PS5, Autonomous Customer Resolution Agent · All data synthetic

Contents

- 01 The problem
- 02 The solution
- 03 Proof it works
- 04 Impact & deployment

01

The problem

Every case looks simple. Almost none of them are.

“My kettle arrived cracked, can you send a replacement by Tuesday? It’s a birthday gift.” No order number. No product code. Finding the answer means checking five systems before anyone can act.

8–15 minutes

of skilled human work,
per routine case, across
five systems.

Hours to first reply

for an answer that could have
been immediate.

Resists automation

the right answer depends on
facts nobody has gathered yet.

Why existing tools fall short

RULES ENGINES

Break the moment reality deviates — they can't tell "stock exists" from "stock exists but arrives too late to matter."

KB CHATBOTS

Can explain the refund policy but can't issue the refund, so the work still lands on a human.

ONE LLM CALL

Can't verify itself, can't recover when an action is refused, and will confidently promise a refund that never posted.

Three people, three different needs

Support agents

Receive a diagnosed case with evidence gathered and the action already taken — or ready in one click.

Support leads

Keep control of the money: approval thresholds, a queue of what needs a human, an audit trail of every action.

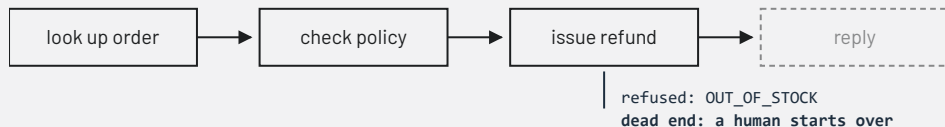
Customers

Get a resolution in seconds instead of a first reply in hours — and a straight answer when it's no.

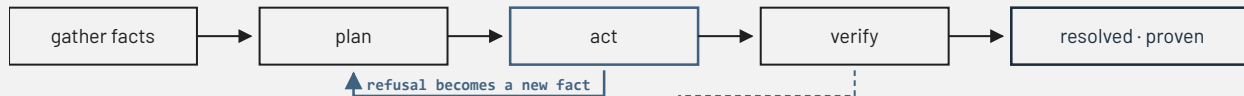
Built on a fictional home-goods retailer, Kestrel Home: nine customers, twelve orders, three warehouses with real transit times, and a nine-policy handbook.

The path is discovered, not planned

A FIXED SCRIPT



AN AGENT



The same refusal ends a script and advances an agent. That single arrow — the failed action re-entering planning as evidence — is the whole difference.

02

The solution

Five specialists behind one state machine

01 Intake	Reads the message and any photo, then binds it to a goal, a customer and an order. Asks one clarifying question when the case is genuinely unworkable.	returns Intake sets priority
02 Investigator	Picks its own tools across CRM, orders, inventory, shipping, the ledger and the handbook, then reports which remedies are actually eligible.	returns Facts get_order · check_inventory · search_policy
03 Resolver	Commits to one remedy and writes a typed action plan that states the outcome it expects, so the result can be checked later.	returns Plan expected outcome
04 Policy Auditor	Re-checks the plan against the handbook on its own. A plan it never approves cannot reach the executor at all.	returns Review search_policy
05 Communicator	Writes the customer reply and the internal note, then files the case as a knowledge-base entry the next investigation can retrieve.	returns Reply writes to the KB

How a request flows

1 Triage	The case arrives at the FastAPI layer from the portal or the desk and Intake reads it.	state: Triage
2 Investigating	Read-only tools run concurrently against the enterprise systems and the policy handbook.	state: Investigating
3 Planning	The Resolver commits to one remedy and names the outcome it expects.	state: Planning
4 Policy review	The Auditor approves, or sends the plan back. The guardrail sends anything over the limit to a human.	state: Policy Review
5 Executing	The Executor runs the typed write actions. A refusal returns as data and planning starts again.	state: Executing
6 Verifying	SQL checks read the systems of record back and compare them with the plan.	state: Verifying
7 Resolved	The reply goes out, the trace is stored and the case becomes a knowledge-base entry.	state: Resolved

Two more roles, and a world that moves

Executor

The only code path permitted to change enterprise state.

The Resolver proposes; it never executes.

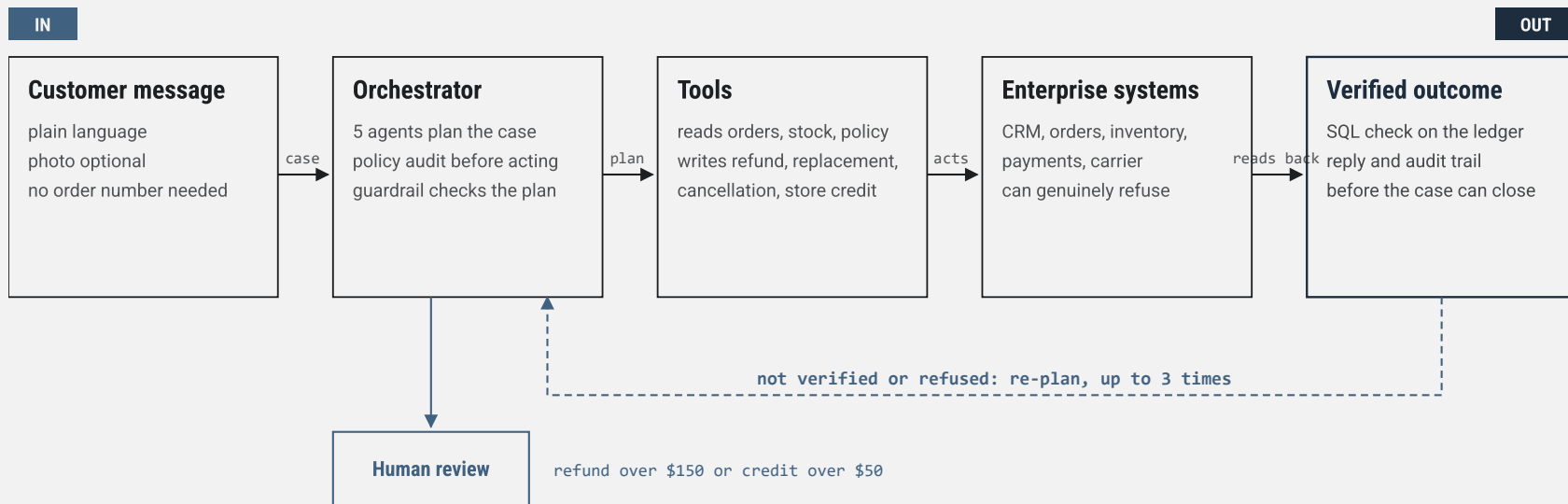
Verifier

Reads the systems of record back and decides whether the case may close.

World events

Conditions change while the agent works, so adaptation is real rather than narrated.

System architecture: one case, end to end



Five stages, left to right. A refused or unverified action sends the case back to planning. A plan over the guardrail threshold waits for a human instead of running.

What sits behind each stage

Read tools

get_order, get_customer, check_inventory, track_shipment, search_policy, search_past_cases and inspect_attachment for the photo. Scoped per agent and run concurrently.

Write actions

refund, cancel_order, create_replacement, create_return_label and issue_store_credit. Typed, and only the Executor may call them.

Memory and models

desk.db keeps the case, the messages and the full agent trace. enterprise.db is the company itself. Bedrock, Groq, Mistral and Gemini in turn, with offline rules beneath.

Two databases on purpose: the agent reaches the company only through tools, and the verifier reads that company directly, so no agent can close a case by claiming success.

When the world moves mid-plan

Case CS-1001. The inventory row really changes between planning and reserving, and the service layer really refuses.

GOAL	Intake binds a message with no order number to C-1001 / ORD-50001 , need-by 2026-09-15
DECISION	Investigator: 6 tool calls → replacement eligible (POL-SLA-1) · refund \$38 eligible (POL-DMG-1)
PLAN	Ship KT-220 from WH-EAST , ETA 2026-09-14, beats the deadline
ENVIRONMENT	The last KT-220 in WH-EAST was sold through the marketplace channel
REFUSED	create_replacement → OUT_OF_STOCK
ADAPT	Re-plan, attempt 2 of 3. WH-WEST has stock, but 6 days transit arrives 2026-09-17, after the deadline
ACTION	Refund \$38 rather than promise something late (POL-DMG-1)
VERIFIED	refund_posted expected \$38, ledger shows \$38 · no_over_refund \$38 of \$70 captured · customer_notified

03

Proof it works

Done is a fact, not a claim

Verified in SQL

Seven checks against the systems of record, including a `no_over_refund` invariant on every case. A failed check re-enters planning. It is not a warning.

Guardrails in code

`Refund > $150, credit > $50, or a fraud flag` freezes the plan for a human. The model is never asked whether approval is needed.

Knowing when to stop

Two of eight demo cases end in not acting: an escalation with evidence, and a clarifying question. Both are correct outcomes.

Eight cases, eight ways the world pushes back

DAMAGED ITEM Last unit sells out mid-plan ADAPTS · refund, verified	REFUND Payment gateway returns 503 RETRIES · same idempotency key	CANCELLATION Carrier collects the order first RE-PLANS · return labels	LATE RETURN 41 days, outside the 30-day window GOODWILL · within tier limit
HIGH-VALUE ITEM \$749 exceeds the auto-approval limit PAUSES · for a supervisor	LOST PACKAGE Tracking 7 days past ETA RE-SHIPS · nearest warehouse	SUSPICIOUS CLAIM Photo proof of delivery + 4 claims in 90 days REFUSES · escalates with evidence	VAGUE REQUEST Two recent orders, no item named ASKS · resumes on the reply

04

Impact & deployment

Human effort moves to where judgement is needed

MEASURE

TODAY

WITH CASEPILOT

Time to first meaningful response

hours

seconds

Handling time, routine case

8–15 min of human work

fully automated, or one approval click

Actions taken without verification

most

none, every case checked against the ledger

Policy consistency

varies by agent and by day

one handbook, independently audited every time

Institutional memory

lives in individual heads

a knowledge-base entry per resolved case

Run it yourself

REPOSITORY	github.com/Parth-sarthidubey/Agentic_Customer_Resolution_CasePilot
HOSTED	Web application on Render, the agent desk and customer portal, live
LOCAL	uv run uvicorn app.main:app --port 8000, no API key required
TESTS	uv run pytest, 10 end-to-end tests, hermetic, every scenario asserted against enterprise state

Stack	Models	Cost to run
FastAPI · SQLite × 2 · vanilla JS desk, no build step · Docker	AWS Bedrock, Groq, Mistral, Gemini: free tiers, with a deterministic fallback beneath	≈ \$0.0025 per case, with a real-time spend cap that disables paid inference when reached

Thank you

CasePilot, PS5, Autonomous Customer Resolution Agent.